

UNIVERSIDAD DE SEVILLA
INSTITUTO DE MICROELECTRÓNICA DE SEVILLA (IMSE-CNM-CSIC)

MÁSTER UNIVERSITARIO EN MICROELECTRÓNICA:
DISEÑO Y APLICACIONES DE SISTEMAS MICRO/NANOMÉTRICOS

**Design of a Key-Generation Cryptographic
Engine for Integrated-Circuit
Implementation: a Transistor-Level
Characterized Entropy Source with FPGA
Verification against an ESP32**

Master's Thesis

Author:
Mario García Jiménez

Supervisor:
[To be assigned]

Academic year 2025–2026

Abstract

Cryptographic keys are only as good as the entropy behind them, yet most embedded key generators are evaluated as black boxes: statistical test suites are run on their output and a pass is taken as evidence of randomness. The author's previous work on the hardware random number generator of the ESP32 showed why that is not enough — a source with low min-entropy can pass NIST SP 800-22 — and NIST itself has since rejected the suite for assessing cryptographic generators. This thesis takes the opposite route: a white-box key-generation engine whose entropy is justified from the physics of the device. A ring-oscillator entropy source is designed at transistor level and its jitter is extracted from simulation and turned, through a validated stochastic model, into a lower bound on min-entropy per bit and into the design parameters of the sampler. Around it, a fully portable digital core — SP 800-90B health tests, an AES-based conditioning and CTR_DRBG with derivation function, AES-CMAC authentication and an I2C register interface — is written once and verified three ways: against the official test vectors, in simulation on a Basys 3 FPGA acting as the verification vehicle, and against an ESP32 that recomputes every result with an independent implementation (mbedtls). The same VHDL is finally synthesized on an open 130 nm standard-cell library to give the area in gate equivalents and the path to a dedicated integrated circuit. All code, models, vectors and data are released.

Keywords true random number generator, ring oscillator, jitter, min-entropy, stochastic model, AIS 20/31, SP 800-90B, CTR_DRBG, AES-CMAC, secure element, FPGA, ASIC, ESP32.

Resumen

Las claves criptográficas valen lo que vale la entropía que hay detrás, y sin embargo la mayoría de los generadores embebidos se evalúan como cajas negras: se pasan baterías de tests estadísticos sobre su salida y un aprobado se toma como prueba de aleatoriedad. El trabajo previo del autor sobre el generador hardware del ESP32 mostró por qué no basta — una fuente con poca min-entropía puede pasar NIST SP 800-22 — y el propio NIST ha rechazado después esa batería para evaluar generadores criptográficos. Este TFM toma el camino contrario: un motor de generación de claves de caja blanca cuya entropía se justifica desde la física del dispositivo. Se diseña a nivel de transistor una fuente de entropía por oscilador de anillo, se extrae su jitter por simulación y se convierte, mediante un modelo estocástico validado, en una cota inferior de min-entropía por bit y en los parámetros de diseño del muestreador. A su alrededor, un núcleo digital totalmente portable — tests de salud SP 800-90B, acondicionado y CTR_DRBG con función de derivación basados en AES, autenticación AES-CMAC e interfaz de registros I2C — se escribe una sola vez y se verifica de tres maneras: contra los vectores oficiales, en simulación sobre una FPGA Basys 3 que hace de vehículo de verificación, y frente a un ESP32 que recalcula cada resultado con una implementación independiente (mbedtls). El mismo VHDL se sintetiza por último sobre una biblioteca abierta de celdas estándar de 130 nm para dar el área en puertas equivalentes y el camino hacia un circuito integrado dedicado. Todo el código, modelos, vectores y datos se publican.

Acknowledgements

To my family, my colleagues at Airzone, and the faculty of the Máster en Microelectrónica at IMSE-CNM, for their support during this work.

Contents

Abstract	1
Resumen	2
Acknowledgements	3
List of Figures	7
List of Tables	8
List of Acronyms	9
1 Introduction	11
1.1 Motivation: keys are only as good as their entropy	11
1.2 From the black-box ESP32 study to a white-box engine	11
1.3 Objectives	11
1.4 Scope and exclusions	11
1.5 Methodology and reproducibility conventions	11
1.6 Structure of the document	11
2 State of the Art	12
2.1 Ring-oscillator TRNG architectures	12
2.2 Stochastic models and entropy bounds	12
2.3 Embedded jitter measurement	12
2.4 Standards: AIS 20/31 v3.0 and NIST SP 800-90B	12
2.5 Symmetric cores for constrained hardware	12
2.6 Commercial secure elements	12
2.7 From FPGA to ASIC: what changes in the entropy source	12
2.8 Gap addressed by this work	12
3 Entropy Source: from Transistor to Min-Entropy	13
3.1 The elementary ring oscillator	13
3.2 Stochastic model and the quality factor	14

3.3	Exact conditional entropy vs. the asymptotic bound	14
3.3.1	Design target	15
3.4	Embedded jitter estimator and its two corrections	15
3.5	Transistor-level bench and its resolution	16
3.6	Design target and divider selection	17
4	Digital Engine	18
4.1	Architecture and the single-primitive decision	18
4.2	AES-128 encryption core	19
4.3	AES-CMAC	19
4.4	CTR_DRBG with derivation function	20
4.5	Health tests	20
4.6	Capture path and clock-domain crossings	21
4.7	I2C register interface and command sequencer	21
5	Verification	23
5.1	Strategy: vectors, simulation, independent verifier	23
5.2	Reference models in Python	23
5.3	Testbenches and results	23
5.4	ESP32 verifier firmware	23
5.5	FPGA bring-up	23
6	Path to an Integrated Circuit	24
6.1	Standard-cell synthesis flow	24
6.2	Area per block in gate equivalents	24
6.3	Optimizations and their expected savings	24
6.4	What the ring oscillator needs in silicon	24
6.5	Cost and schedule of an academic shuttle	24
7	Results and Comparison	25
7.1	Entropy campaign on the FPGA	25
7.2	Comparison with the ESP32 generator	25
7.3	Comparison with commercial secure elements	25
7.4	Uncertainty budget	25
8	Conclusions and Future Work	26
8.1	Conclusions	26
8.2	Contributions	26
8.3	Future work	26
	Bibliography	27

A	Register Map	30
B	Official Test Vectors Used	31
C	Reproduction Guide	32

List of Figures

4.1	Data path of the engine. Only the ring oscillators are technology dependent; every other block is portable RTL.	18
-----	---	----

List of Tables

3.1	Conditional Shannon entropy per bit: exact wrapped-Gaussian computation vs. truncated expansions [SIMULATED].	15
3.2	Recovery of a known Q_{raw} by the corrected embedded estimator ($K = 20\,000$ blocks of $N = 100$ pairs) [SIMULATED].	16

List of Acronyms

AES	Advanced Encryption Standard
AIS	Anwendungshinweise und Interpretationen zum Schema (BSI)
APT	Adaptive Proportion Test
ASIC	Application-Specific Integrated Circuit
BSI	Bundesamt für Sicherheit in der Informationstechnik
CAVP	Cryptographic Algorithm Validation Program
CDC	Clock Domain Crossing
CMAC	Cipher-based Message Authentication Code
CTR_DRBG	Counter-mode Deterministic Random Bit Generator
DRBG	Deterministic Random Bit Generator
ERO	Elementary Ring Oscillator
FPGA	Field-Programmable Gate Array
GE	Gate Equivalent
I2C	Inter-Integrated Circuit
KAT	Known-Answer Test
LUT	Look-Up Table
MEMS	Micro-Electro-Mechanical System
MPW	Multi-Project Wafer
NIST	National Institute of Standards and Technology
PDK	Process Design Kit

PLL	Phase-Locked Loop
PUF	Physically Unclonable Function
RCT	Repetition Count Test
RO	Ring Oscillator
RTL	Register-Transfer Level
SPI	Serial Peripheral Interface
TRNG	True Random Number Generator
XADC	Xilinx Analog-to-Digital Converter

Chapter 1

Introduction

- 1.1 Motivation: keys are only as good as their entropy
- 1.2 From the black-box ESP32 study to a white-box engine
- 1.3 Objectives
- 1.4 Scope and exclusions
- 1.5 Methodology and reproducibility conventions
- 1.6 Structure of the document

Chapter 2

State of the Art

- 2.1 Ring-oscillator TRNG architectures
- 2.2 Stochastic models and entropy bounds
- 2.3 Embedded jitter measurement
- 2.4 Standards: AIS 20/31 v3.0 and NIST SP 800-90B
- 2.5 Symmetric cores for constrained hardware
- 2.6 Commercial secure elements
- 2.7 From FPGA to ASIC: what changes in the entropy source
- 2.8 Gap addressed by this work

Chapter 3

Entropy Source: from Transistor to Min-Entropy

This chapter builds the chain that the rest of the thesis rests on: from the physical jitter of a ring oscillator to a defensible figure of min-entropy per output bit, and from that figure to the design parameters of the sampler. Every step is validated before it is used. The statistical estimators are validated against synthetic data of known jitter; the analytic bounds are validated against an exact numerical computation; and the transistor-level bench is validated by measuring its own resolution before anything is measured with it. All figures in this chapter are [SIMULATED] unless stated otherwise; the hardware campaign is reported in Chapter 7.

3.1 The elementary ring oscillator

The entropy source is an Elementary Ring Oscillator (ERO): a free-running ring oscillator RO1 whose output is sampled by a flip-flop clocked by a second ring oscillator RO2 divided by an integer K_D [1, 2]. Between two samples the phase of RO1 accumulates jitter for K_D periods of RO2; when the accumulated jitter is comparable to the period of RO1, the sampled bit becomes unpredictable. The ERO was chosen over higher-throughput alternatives (multi-ring XOR, coherent sampling, transition-effect rings) for one reason: it is the architecture for which a stochastic model with a provable entropy bound exists and for which an embedded jitter measurement method has been published and validated [3]. Throughput is not a requirement here — the engine consumes entropy only to seed a deterministic generator — whereas a demonstrable bound is.

A design decision that follows from the platform is that *the board oscillator is never used as a reference*. The Basys 3 clock source is a MEMS oscillator with a fractional Phase-Locked Loop (PLL) whose datasheet specifies a cycle-to-cycle jitter of up to 95 ps [ESTIMATED] [4], one to two orders of magnitude above the per-period jitter of a ring

oscillator in a 28 nm process (2–4 ps [ESTIMATED] [3]). Sampling RO1 with that clock would measure the phase noise of the PLL, a global and potentially inducible component, rather than the thermal noise of the ring. RO2 therefore samples RO1, and the 100 MHz system clock only drives the synchronous logic.

3.2 Stochastic model and the quality factor

Following Baudet *et al.* [1], the phase $\varphi(t)$ of RO1, measured in periods, is modeled as a Wiener process with drift μ and volatility σ^2 . Between two samples separated by Δt the phase advances $\nu = \mu \Delta t$ periods on average and accumulates a variance

$$Q = \sigma^2 \Delta t, \quad (3.1)$$

the *quality factor*. The output bit is $s = 1$ when the fractional phase lies in $[1/2, 1)$. Conditioned on the previous phase φ_0 , the bit is Bernoulli with

$$P(s = 1 \mid \varphi_0) = \sum_{k \in \mathbb{Z}} \left[\Phi\left(\frac{k+1-\varphi_0-\nu}{\sqrt{Q}}\right) - \Phi\left(\frac{k+\frac{1}{2}-\varphi_0-\nu}{\sqrt{Q}}\right) \right], \quad (3.2)$$

where Φ is the standard normal distribution function; this is the wrapped Gaussian evaluated exactly. Its first harmonic gives the maximum bias $\varepsilon_{\max} \approx (2/\pi) e^{-2\pi^2 Q}$, from which Baudet’s Corollary 1 yields the well-known expression

$$H(s \mid \varphi_0) = 1 - \frac{4}{\pi^2 \ln 2} e^{-4\pi^2 Q} + O\left(e^{-6\pi^2 Q}\right). \quad (3.3)$$

3.3 Exact conditional entropy vs. the asymptotic bound

Equation (3.3) is routinely cited as a *lower bound* on entropy per bit. Read carefully, the inequality in the corollary is only that the entropy rate of the sequence is at least the entropy of one bit conditioned on the exact previous phase; the right-hand side is an asymptotic expansion *of* that conditional entropy, not a bound on it. Expanding the binary entropy in the bias, $h_2(\frac{1}{2} + \varepsilon) = 1 - \frac{1}{\ln 2} \sum_{n \geq 1} \frac{(2\varepsilon)^{2n}}{2n(2n-1)}$, and averaging over a uniform φ_0 with $\varepsilon = A \sin(2\pi\varphi)$, $A = (2/\pi)e^{-2\pi^2 Q}$, gives

$$H(s \mid \varphi_0) = 1 - \frac{4}{\pi^2 \ln 2} e^{-4\pi^2 Q} - \frac{8}{\pi^4 \ln 2} e^{-8\pi^2 Q} - \dots \quad (3.4)$$

All omitted terms are negative, so *any* truncation overestimates the entropy; the next term decays as $e^{-8\pi^2 Q}$, not $e^{-6\pi^2 Q}$. Table 3.1 compares the exact value computed from (3.2) on a grid of 8192 phases with the first- and second-order expansions. Adding the second-order term reduces the error of (3.3) by a factor of 95 at $Q = 0.10$ and 4970 at

$Q = 0.20$, which confirms the derivation.

Table 3.1: Conditional Shannon entropy per bit: exact wrapped-Gaussian computation vs. truncated expansions [SIMULATED].

Q	Exact	Order 2	Order 1 (Eq. 3.3)	Error of order 1
0.02	0.7013194	0.7100952	0.7345213	$+3.3 \times 10^{-2}$
0.05	0.9163001	0.9164920	0.9187783	$+2.5 \times 10^{-3}$
0.10	0.9886728	0.9886733	0.9887174	$+4.5 \times 10^{-5}$
0.15	0.9984319	0.9984319	0.9984327	$+8.5 \times 10^{-7}$
0.20	0.9997823	0.9997823	0.9997823	$+1.6 \times 10^{-8}$

In the operating region ($Q \approx 0.2$) the discrepancy is 10^{-8} and irrelevant; for an under-dimensioned generator ($Q \approx 0.02$) the truncated formula credits 3.3 % of entropy that does not exist. This thesis therefore uses the exact computation as the conservative value and cites (3.3) as what it is: an excellent asymptotic approximation for $Q \gtrsim 0.1$. The same numerical machinery shows that the min-entropy obtained from the first-harmonic bias, $H_\infty = -\log_2(\frac{1}{2} + \varepsilon_{\max})$, agrees with the exact worst-case value to better than 10^{-4} bit over $Q \in [0.05, 0.30]$.

3.3.1 Design target

AIS 20/31 version 3.0 [5] admits two alternative criteria for class PTG.2: Shannon entropy ≥ 0.9998 or min-entropy ≥ 0.98 per internal bit. Solving both by bisection on the exact entropies gives $Q \geq 0.2022$ and $Q \geq 0.2286$ respectively. The min-entropy criterion is the more demanding one and is also the quantity NIST SP 800-90B works with, so it is adopted here: **the design target is $Q \geq 0.23$, with a set point of $Q = 0.30$** ($H_\infty = 0.9951$), the margin covering the reduction of thermal jitter at low temperature, which is the standard adversarial test of Fischer and Lubicz [2].

3.4 Embedded jitter estimator and its two corrections

To choose K_D one must know σ^2 , and the method of Fischer and Lubicz [2] measures it from the bits themselves: with $K_D = 1$, the fraction c of pairs (b_j, b_{j+M}) that differ within a block of N pairs estimates the phase accumulated over M samples, and its variance across K blocks grows as $4MQ_{\text{raw}}$. Implemented literally, with a straight-line fit, the estimator *underestimates* Q_{raw} by 13 % to 38 % depending on the jitter level. Two causes were identified on synthetic data with known Q and corrected:

1. **Vertex effect.** c is not linear in the accumulated phase; it is a *triangular* wave of period one and slope ± 2 . When the deterministic phase $M\nu \bmod 1$ falls near

a vertex the distribution straddles it and the variance collapses — to 0.38 of its nominal value in one measured case. The vertex is detected from the measured mean itself ($\bar{c} \approx 0$ or 1 is a vertex, $\bar{c} \approx \frac{1}{2}$ the middle of the ramp), so distances M with $\bar{c}$ outside $[0.15, 0.85]$, or for which $3\sqrt{MQ}$ reaches the vertex, are discarded iteratively.

2. **Non-constant estimation noise.** Estimating a fraction from N pairs adds a variance that scales with $\bar{c}(1 - \bar{c})$, which changes with M . Fitting it as a constant intercept biases the slope; it is fitted instead as a second regressor, $\text{Var}(c) = Q(4M) + \kappa \bar{c}(1 - \bar{c})/N$.

After both corrections the estimator recovers the injected Q_{raw} within $\pm 2\%$ over a range of $150\times$ (Table 3.2), with a bias of -0.6% and a dispersion of 4.0% over eight independent seeds for $K = 20\,000$ blocks. The fitted accumulation exponent is 1.009 for a simulator without flicker noise; measured on hardware, the same exponent separates the thermal component ($b = 1$, which counts as entropy) from flicker and global jitter ($b = 2$, which does not).

Table 3.2: Recovery of a known Q_{raw} by the corrected embedded estimator ($K = 20\,000$ blocks of $N = 100$ pairs) [SIMULATED].

Q_{raw} injected	Estimated	Error	M values used
2.0×10^{-7}	1.979×10^{-7}	-1.07%	19
5.0×10^{-7}	5.060×10^{-7}	$+1.21\%$	18
2.0×10^{-6}	2.037×10^{-6}	$+1.86\%$	16
8.0×10^{-6}	8.039×10^{-6}	$+0.48\%$	9
3.0×10^{-5}	2.977×10^{-5}	-0.78%	5

A practical consequence for the hardware is that the blocks need not be contiguous: each requires only $N + M$ consecutive bits, and independent blocks give a slightly better estimate (-0.30% error with 20 000 independent chunks). The capture buffer in the FPGA therefore holds a single chunk of a few kilobits, filled at ring speed and read out slowly, instead of megabits of contiguous samples.

3.5 Transistor-level bench and its resolution

The entropy source is the only block of the engine tied to the technology, and in an integrated circuit it is designed at transistor level. A SPICE bench was built around a five-stage ring with an enabling NAND gate, with every technology-dependent parameter (device models, widths, minimum length, supply) confined to a header so that moving from the generic placeholder models used here to a foundry Process Design Kit (PDK) changes one block of the netlist. Jitter is extracted from the threshold-crossing instants

of the output by the second difference $D(M) = t_{k+2M} - 2t_{k+M} + t_k$, i.e. the Allan variance applied to phase data, whose expectation $2M\sigma^2$ is free of the bias that a simple difference over overlapping windows carries (measured at 5%), and which is the method used by Benea *et al.* to characterize rings both in Artix-7 and in 28 nm silicon [6]. Estimates at different M are combined with inverse-variance weights proportional to the number of independent windows; without that weighting the dispersion stayed at 15% regardless of the simulated length.

The bench measures its own resolution first: with all noise sources disabled, the residual jitter is the numerical noise of the simulator. The first attempt gave 0.33 ps, of the same order as the jitter to be measured; it was entirely the start-up transient of the oscillator, and discarding the first twenty cycles brings the floor to **1.38 fs**, three orders of magnitude below the signal. Only then was noise injected. Jitter scales with the square root of the injected spectral density ($\times 2.01$ measured for $\times 4$ in density, against $\times 2.00$ expected) up to the point where the r.m.s. noise current reaches about 80% of the stage drive current; there the small-signal regime breaks down, the mean period shifts, and the scaling law fails by a factor of five. A validity criterion follows for every campaign, including those on a real PDK: keep injected noise below $\sim 40\%$ of the stage current and verify that the mean period does not move when noise is enabled. With the placeholder models the ring oscillates at 4.3 GHz, which is not physical; the numbers that enter the thesis come from the foundry models, where transient noise analysis injects the device noise itself.

3.6 Design target and divider selection

With σ/T measured per period, $Q_{\text{raw}} = (\sigma/T)^2$ and the divider follows from $K_D = \lceil Q_{\text{target}}/Q_{\text{raw}} \rceil$. For the range of jitter expected in a 28 nm process the divider lies between 3×10^4 and 5×10^5 and the throughput per ring between 1 and 20 kbit/s. The model reproduces the published design point of Fischer and Lubicz ($K_D \approx 430\,000$ for $\sigma = 5.01$ ps over $T = 7.81$ ns) [2]. Such rates are sufficient here: seeding the deterministic generator with 256 bits of full entropy requires about 512 raw bits at $H_\infty \approx 0.98$, i.e. between 0.03 and 0.5 s per seed, and the engine issues a seed only at start-up and at reseed intervals. If more throughput were ever needed, the clean route is several independent EROs, each with its own bound, rather than a multi-ring XOR whose independence cannot be demonstrated [7].

Chapter 4

Digital Engine

The digital engine turns raw bits from the entropy source into keys and uses them, and it is written so that the same Register-Transfer Level (RTL) runs on the FPGA used for verification and, later, on standard cells. Nothing in it is tied to a vendor: no primitives, no memory macros, no clock-management blocks. The only technology-dependent block of the whole design is the pair of ring oscillators of Chapter 3.

4.1 Architecture and the single-primitive decision

Figure 4.1 summarizes the data path. Raw bits leave the ERO in the clock domain of the sampling ring, cross to the system clock one bit at a time, and feed three consumers: the health tests, the seed collector of the deterministic generator, and a capture buffer used only for characterization. Keys come out of a Counter-mode Deterministic Random Bit Generator (CTR_DRBG); the current key is loaded into an AES-CMAC block that answers challenges; and an Inter-Integrated Circuit (I2C) slave exposes everything as a register map.

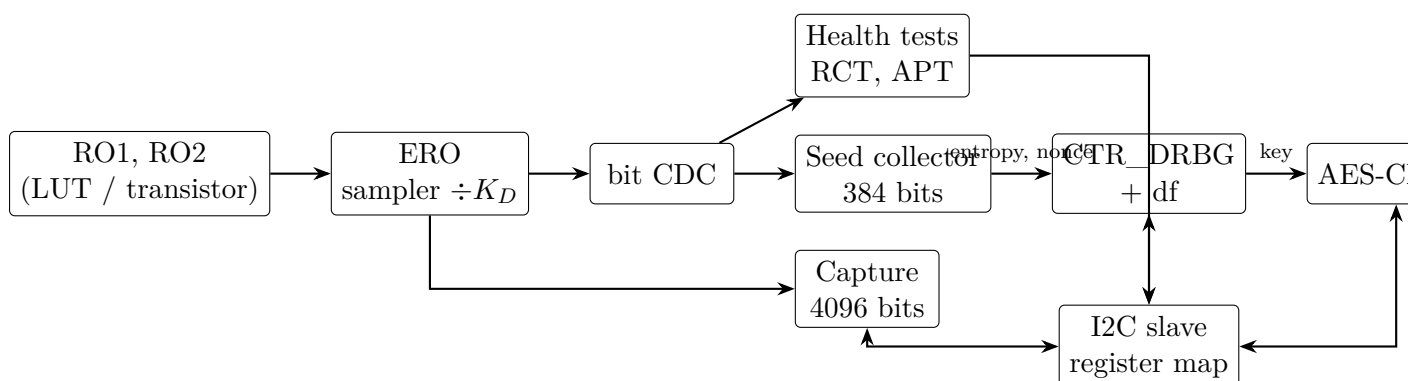


Figure 4.1: Data path of the engine. Only the ring oscillators are technology dependent; every other block is portable RTL.

The decision that shapes the whole engine is to use **one cryptographic primitive**,

AES-128, for the three jobs: conditioning the entropy, generating keys and authenticating. The first architecture had a SHA-256 core as conditioner; the synthesis estimates of Chapter 6 showed that it would take almost half the digital area, and SP 800-90B lists AES-based constructions — CMAC, CBC-MAC and the block-cipher derivation function of SP 800-90A — among its *vetted* conditioning components [8]. Dropping the hash brings the engine from an estimated ~ 25 to ~ 13 Gate Equivalents (GEs) (thousands), at the price of moving from a CTR_DRBG *without* derivation function, which needs full-entropy input, to one *with* it, which is the common configuration in practice and accepts non-full-entropy input directly. The AES core is encryption-only: counter mode and CMAC never decrypt, and offering decryption as a service to the ESP32 would be pointless since the ESP32 has its own accelerator. Cross-verification is unaffected: the engine encrypts and the ESP32 decrypts with mbedtls.

4.2 AES-128 encryption core

The core is iterative, one round per clock cycle, eleven cycles per block (initial key addition plus ten rounds), with the round keys expanded on the fly from the previous one — forward expansion is all that encryption needs, and it saves storing eleven round keys. At 100 MHz that is 1.16 Gbit/s of core throughput, far above what any I2C link can carry, so pipelining would be wasted area; the iterative form is chosen instead because one round is a pure combinational function that can be checked directly against the standard [9]. The S-box is a 256-entry table synthesized as logic, MixColumns is written as the usual `xtime` network, and the state follows the column-major convention of FIPS 197 with byte 0 on the left, so that hexadecimal strings in the standard map one-to-one onto the VHDL vectors.

The core was verified against the five official vectors available for AES-128 encryption (FIPS 197 appendices B and C.1, and the three ECB blocks of SP 800-38A F.1.1) and against a chain of one thousand encryptions to expose any state left over between blocks. A check that had not been planned turned out to be the most convincing: the encryption of the all-zero block under the SP 800-38A example key gave `7df76b0c...546f`, which is exactly the intermediate value L published by NIST in its CMAC examples [10].

4.3 AES-CMAC

AES-CMAC [11, 12] is the primitive of the challenge-response: the ESP32 sends a nonce, the engine returns its tag under the current key. The block computes the sub-keys K_1 , K_2 once when a key is loaded (one encryption of the zero block and two conditional left shifts with the polynomial constant `0x87`) and then runs CBC-MAC over the message, applying K_1 to a complete last block or padding with `0x80` and applying K_2 otherwise.

It was chosen over HMAC-SHA-256 because it reuses the only primitive and costs one encryption per block against two hash compressions. The block passes the four vectors of RFC 4493 (empty message, one block, 40 and 64 bytes), which exercise both padding paths.

4.4 CTR_DRBG with derivation function

The generator implements CTR_DRBG with AES-128 and the block-cipher derivation function of SP 800-90A [13]. Its input format is fixed on purpose: 256 bits of raw entropy and 128 bits of nonce, so that $S = L \parallel N \parallel E \parallel \text{nonce} \parallel 0x80 \parallel 0 \dots$ is exactly four blocks and the derivation function becomes a closed sequence of fourteen encryptions; a variable-length input would cost a byte-oriented datapath for no benefit. Instantiation (or reseed) costs 14 AES operations, 183 cycles; generating 256 bits costs four, 53 cycles. The state (Key, V) is never exposed; the reseed counter is readable so that the host can enforce a reseed policy far below the 2^{48} limit of the standard.

Verification proceeded in two steps. First, a bit-exact reference model was written in Python — with its own AES, independent from the VHDL — and run against the CAVP vectors: 960 AES-128 cases with and without derivation function, with and without reseed, all passing. Second, that model generated the expected outputs for the fixed input layout of the hardware, and the VHDL reproduced them: two consecutive generations, a reseed followed by a generation, the reseed counter, and the fact that a new instantiation erases the previous state.

4.5 Health tests

The two tests approved by SP 800-90B run continuously on the raw bits [8]. The Repetition Count Test fails when $C_{\text{RCT}} = 1 + \lceil -\log_2 \alpha / H \rceil$ identical samples arrive in a row; the Adaptive Proportion Test counts, in a window of $W = 1024$ samples (the binary-source value; 512 is for non-binary sources), how many repeat the first one, and fails at $C_{\text{APT}} = 1 + \text{CRITBINOM}(W, 2^{-H}, 1 - \alpha)$. For the design target $H = 0.98$ and $\alpha = 2^{-20}$ these are $C_{\text{RCT}} = 22$ and $C_{\text{APT}} = 596$; both are generics, because H is set from the *measured* min-entropy, not assumed. The proportion test also checks the complement of the count, which the standard permits for binary sources: a source stuck at the value opposite to the window’s first sample would otherwise score zero and pass. Alarms are sticky until explicitly cleared, and while any alarm is raised the sequencer refuses to seed.

The testbench injects the failure modes the tests exist for: a healthy source passes twenty windows (worst run 13, worst proportion 540); a stuck source trips the repetition test at exactly sample 22; a source biased to 70 % trips the proportion test at 713 without tripping the repetition test; and a source stuck at the complement is caught. The first

“healthy” source of the bench — a shift register seeded with a single one — was itself caught by the tests, with a start-up run of 27 zeros and a first window at 76 %. The detector was right and the bench was wrong; the source now starts from a dense seed and is warmed up.

4.6 Capture path and clock-domain crossings

Raw bits are produced in the clock domain of RO2 and consumed in the system domain. The bit-serial path to the health tests and the seed collector uses a toggle synchronizer: the source registers the bit and inverts a flag; the destination synchronizes the flag through three flip-flops and reads the bit on the flag’s edge. It requires each bit to last at least eight system cycles, i.e. $K_D \geq 64$ for a ~ 700 MHz sampling ring at 100 MHz, which the generator mode satisfies by orders of magnitude. The characterization path does not cross bit by bit: the capture buffer is written in the ring domain at full rate into a 4096-bit memory and read out in the system domain afterwards, with the two directions of the handshake carried by toggle flags. Writing and reading never overlap, so no asynchronous FIFO is needed, and the buffer size follows from Section 3.4: one chunk of $N + M_{\max}$ bits is all the estimator needs.

4.7 I2C register interface and command sequencer

The product interface is I2C, as in commercial secure elements: two pins, addressable, and the bus the ESP32 uses for its peripherals. The slave implements the register-pointer protocol with auto-increment, a five-sample glitch filter on both lines as the specification requires, and open-drain outputs. It does not stretch the clock: at 400 kHz there are 250 system cycles per bit, plenty for any register access, and long operations (seeding, generation) are issued as commands and polled through a status register, which is how commercial devices behave. The slave was tested with a bus-functional master; two real defects surfaced — the pointer did not auto-increment on multi-byte writes, and the most significant bit of each read byte was driven one clock edge late, so every read would have been shifted — and were fixed.

The register map (Appendix A) exposes identification, status and control, the divider and capture-page configuration, the health-test diagnostics, the reseed counter, the challenge and its tag, the last 256 bits generated and, *only while a test pin is asserted*, the current key and the capture buffer. In the FPGA the test pin is a switch; in an integrated circuit it would be a fuse blown at production, because exposing either the key or the raw entropy is a vulnerability. A small sequencer serializes the commands: seeding collects 384 raw bits, aborts if a health alarm rises meanwhile, and instantiates or reseeds the

generator; generation takes 256 bits, makes the first 128 the key and reloads the CMAC sub-keys; authentication runs the CMAC over the challenge block.

The whole engine was exercised as the ESP32 will exercise it: a bus-functional I2C master reads the identification, configures the divider, seeds, generates, reads the key in test mode, writes two challenges and reads their tags, generates again, captures raw bits and verifies that the key is unreadable without the test pin — sixteen checks. What a VHDL bench cannot know is whether the tags are *correct*, because the key came from ring-oscillator noise; the bench dumps key, challenges and tags to a file and a Python script recomputes the CMACs with the independent AES. They match. That is the same cross-check the ESP32 performs on hardware with mbedtls.

Chapter 5

Verification

- 5.1 Strategy: vectors, simulation, independent verifier
- 5.2 Reference models in Python
- 5.3 Testbenches and results
- 5.4 ESP32 verifier firmware
- 5.5 FPGA bring-up

Chapter 6

Path to an Integrated Circuit

- 6.1 Standard-cell synthesis flow
- 6.2 Area per block in gate equivalents
- 6.3 Optimizations and their expected savings
- 6.4 What the ring oscillator needs in silicon
- 6.5 Cost and schedule of an academic shuttle

Chapter 7

Results and Comparison

7.1 Entropy campaign on the FPGA

7.2 Comparison with the ESP32 generator

7.3 Comparison with commercial secure elements

7.4 Uncertainty budget

Chapter 8

Conclusions and Future Work

8.1 Conclusions

8.2 Contributions

8.3 Future work

Bibliography

- [1] M. Baudet, D. Lubicz, J. Micolod, and A. Tassiaux, “On the security of oscillator-based random number generators,” *Journal of Cryptology*, vol. 24, no. 2, pp. 398–425, 2011, [VERIFICADO] ePrint 2009/299 leído; Crossref confirma DOI/vol/págs. $Q = \sigma^2 \Delta t$; Prop. 1 ecs. 8, 10, 12; Cor. 1 ec. 14. [Online]. Available: <https://eprint.iacr.org/2009/299.pdf>
- [2] V. Fischer and D. Lubicz, “Embedded evaluation of randomness in oscillator based elementary TRNG,” in *Cryptographic Hardware and Embedded Systems – CHES 2014*, ser. LNCS, vol. 8731. Springer, 2014, pp. 527–543, [VERIFICADO] PDF completo; Algoritmo 1; error $< 5\%$; $\sigma = 5.01$ ps / 7.81 ns en Cyclone III; $K_D \approx 430\,000$ para $H \geq 0.997$. [Online]. Available: <https://www.iacr.org/archive/ches2014/87310109/87310109.pdf>
- [3] O. Petura, U. Mureddu, N. Bochard, V. Fischer, and L. Bossuet, “A survey of AIS-20/31 compliant TRNG cores suitable for FPGA devices,” in *26th International Conference on Field Programmable Logic and Applications (FPL 2016)*, 2016, pp. 1–10, [VERIFICADO] PDF HAL completo + Crossref; Tabla II (Spartan-6, Cyclone V, SmartFusion2); jitter 2–4 ps; ec. (1) H_{min} . [Online]. Available: <https://hal.science/hal-01382955/document>
- [4] Digilent Inc., *Basys3 FPGA Board Reference Manual (Revised August 12, 2014; applies to Basys3 rev. C)*, Digilent, 2014, xC7A35T-1CPG236C; 33 280 celdas lógicas en 5200 slices (4 LUT6 + 8 FF por slice); oscilador de 100 MHz en W5; Pmod JA/JB/JC + JXADC, 3,3 V en pines 6 y 12. Versión web actual: <https://digilent.com/reference/programmable-logic/basys-3/reference-manual> (no accesible en esta sesión). [VERIFICADO: PDF extraído]. [Online]. Available: https://www.amd.com/content/dam/amd/en/documents/university/aup-boards/XUPBasys3/documentation/Basys3_rm_8_22_2014.pdf
- [5] M. Peter and W. Schindler, “A proposal for functionality classes for random number generators, version 3.0,” Bundesamt für Sicherheit in der Informationstechnik (BSI), Mathematical-technical reference of AIS 20/31, Sep. 2024, [VERIFICADO] PDF 306 pp.; PTG.2.2: Shannon ≥ 0.9998 y/o min-entropía ≥ 0.98 , Prob(1)

- en (0.493,0.507); §4.5.3 online test, §4.5.4 total failure test, §4.6 suite T_{irr} . [Online]. Available: https://www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Certification/Interpretations/AIS_31_Functionality_classes_for_random_number_generators_e_2024.pdf
- [6] L. Benea, M. Carmona, V. Fischer, F. Pebay-Peyroula, and R. Wacquez, “Impact of the flicker noise on the ring oscillator-based TRNGs,” in *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, 2024, [VERIFICADO: PDF leído por el investigador; DOI pendiente de anotar. Datos y scripts abiertos en github.com/opentrng/papers].
- [7] N. Bochard, F. Bernard, V. Fischer, and B. Valtchanov, “True-randomness and pseudo-randomness in ring oscillator-based true random number generators,” *International Journal of Reconfigurable Computing*, vol. 2010, pp. Article ID 879 281, 1–13, 2010, [VERIFICADO-META] Crossref.
- [8] M. S. Turan, E. Barker, J. Kelsey, K. A. McKay, M. L. Baish, and M. Boyle, “Recommendation for the entropy sources used for random bit generation,” NIST, Special Publication SP 800-90B, Jan. 2018, [VERIFICADO: PDF extraído; sección 3.1.5 completa (lista de componentes vetted, Tabla 1, fórmula Output_Entropy, factor 0,999 para no vetted). Autores citados de memoria]. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/90/b/final>
- [9] National Institute of Standards and Technology, “Advanced encryption standard (AES),” NIST, Federal Information Processing Standards Publication FIPS 197-upd1, May 2023, publicado originalmente el 26-11-2001; actualización editorial (sin cambios técnicos) del 9-05-2023. [VERIFICADO: página CSRC y PDF, Tabla 3 y Apéndice D]. [Online]. Available: <https://csrc.nist.gov/pubs/fips/197/final>
- [10] —, “Block cipher modes of operation — CMAC mode for authentication: Example values (CMAC-AES128/192/256),” https://csrc.nist.gov/CSRC/media/Projects/Cryptographic-Standards-and-Guidelines/documents/examples/AES_CMAC.pdf, 2016, ejemplos retirados del Apéndice D de SP 800-38B en 2016. [VERIFICADO: PDF extraído; K1/K2 y etiquetas coinciden con RFC 4493].
- [11] M. Dworkin, “Recommendation for block cipher modes of operation: The CMAC mode for authentication,” NIST, Special Publication SP 800-38B (incl. updates as of 10-06-2016), Oct. 2016, publicado en mayo de 2005; erratas sustantivas del 6-10-2016 (frontal, retirada de ejemplos del Ap. D, bibliografía). [VERIFICADO: PDF extraído; autor según portada del PDF]. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/38/b/upd1/final>

- [12] J. Song, R. Poovendran, J. Lee, and T. Iwata, “The AES-CMAC algorithm,” IETF, RFC 4493, Jun. 2006, [VERIFICADO: vectores de prueba de la sección 4 confirmados; autores y fecha citados de memoria, NO VERIFICADOS en esta sesión]. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc4493>
- [13] E. Barker and J. Kelsey, “Recommendation for random number generation using deterministic random bit generators,” NIST, Special Publication SP 800-90A Rev. 1, Jun. 2015, [VERIFICADO: PDF extraído; Tablas 2 y 3, secciones 8.6, 10.1, 10.2.1.3.1 y 11 leídas. Autores citados de memoria]. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/90/a/r1/final>

Appendix A

Register Map

Appendix B

Official Test Vectors Used

Appendix C

Reproduction Guide